



Fullstack Web Development

Session 7 | Week 7/Day 2 | 120 min

React JS - Part 03 - END

Instructor: Ms. Liev Nova



090667393



What is Data Flow in React?

Data Flow describes **how data moves** inside a React application.

In React:

- API → State → UI → User Action → API → State → UI

React follows **one-way data flow**:

- Data comes **from state**
- UI displays state
- User actions change state
- React re-renders automatically

Data Flow in React

Example: Student Management App

- Features:
 - Load students
 - Display students
 - Add new student
 - Update UI automatically

Data Flow in React

1. Component Mounts (Load Data)

- When the page loads, React needs to **fetch students from an API**.
 - We use **useEffect**.
 - **Example API:** <https://jsonplaceholder.typicode.com/users>

- Fetch Students on Page Load

- **What Happens Here?**

- Component loads
- useEffect runs once
- API request is sent
- Data stored in state
- UI updates automatically

1

```
import { useEffect, useState } from "react";

function StudentsPage() {
  const [students, setStudents] = useState([]);
  const [loading, setLoading] = useState(false);
  const [errorMessage, setErrorMessage] = useState("");
```

3

```
    loadStudents();

    return () => {
      isMounted = false;
    };
  }, []);
```

2

```
useEffect(() => {
  let isMounted = true;

  const loadStudents = async () => {
    try {
      setLoading(true);

      const response = await fetch(
        "https://jsonplaceholder.typicode.com/users"
      );

      const data = await response.json();

      if (!isMounted) return;

      setStudents(data);
      setErrorMessage("");
    } catch (error) {
      setErrorMessage(error.message);
    } finally {
      setLoading(false);
    }
  };
```

Data Flow in React

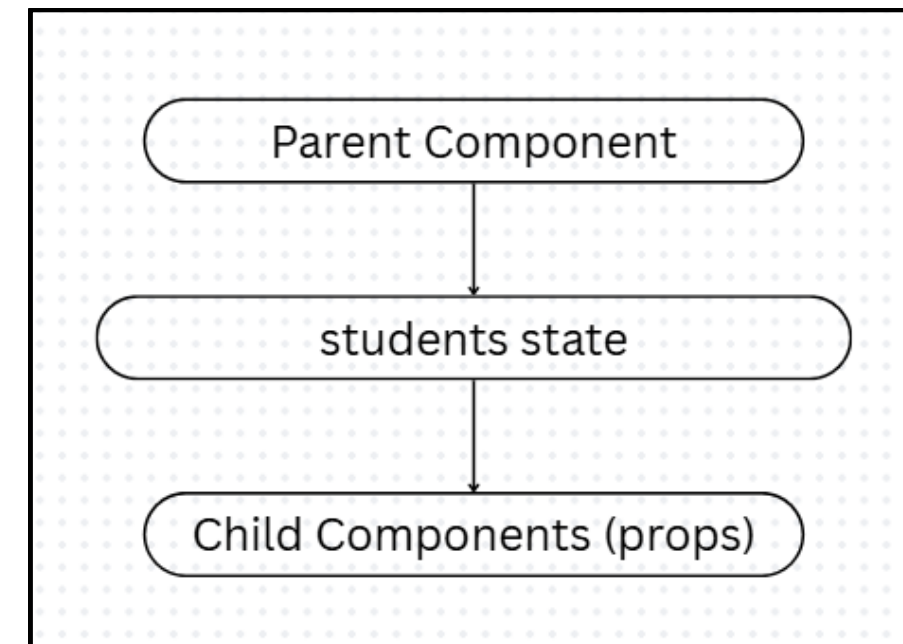
2. Display Data (Render UI)

- React renders components using the stored state.

```
return (  
  <div>  
    <StudentForm onSubmit={createStudent} />  
  
    <StudentsTable  
      students={students}  
      onDelete={deleteStudent}  
    />  
  </div>  
);
```

4

- **Data Flow Here**



- State stays in parent
- Children receive data via **props**

Data Flow in React

3. User Action

- User interacts with the UI.
 - Example:
 - Fill form
 - Click **Add Student**
 - Name: Dalii
 - Gender: Female
 - Year: 2006
 - This triggers a function.

```
const handleSubmit = async (e) => {  
  e.preventDefault();  
  
  const newStudent = {  
    name: currentName.trim(),  
    gender: currentGender,  
    year_of_birth: currentYear  
  };  
  
  await createStudent(newStudent);  
};
```



Add New Student

Name (Latin)	Gender	Year of Birth
Student name	Male	1999
Add Student		

Add New Student

Name (Latin)	Gender	Year of Birth
Dalii	Female	2006
Add Student		

Data Flow in React

4. API Call (Send Data to Server)

- Now React sends data to an API using **POST**.
- **What Happens?**
 - Form sends data
 - POST request sent
 - Server saves data
 - New student returned
 - State updated

```
const createStudent = async (student) => {
  try {
    const response = await fetch(
      "https://jsonplaceholder.typicode.com/users",
      {
        method: "POST",
        headers: {
          "Content-Type": "application/json"
        },
        body: JSON.stringify(student)
      }
    );

    const data = await response.json();

    setStudents((prev) => [...prev, data]);
  } catch (error) {
    alert("Failed to add student");
  }
};
```

Data Flow in React

5. Refresh State

- Instead of reloading the whole page:

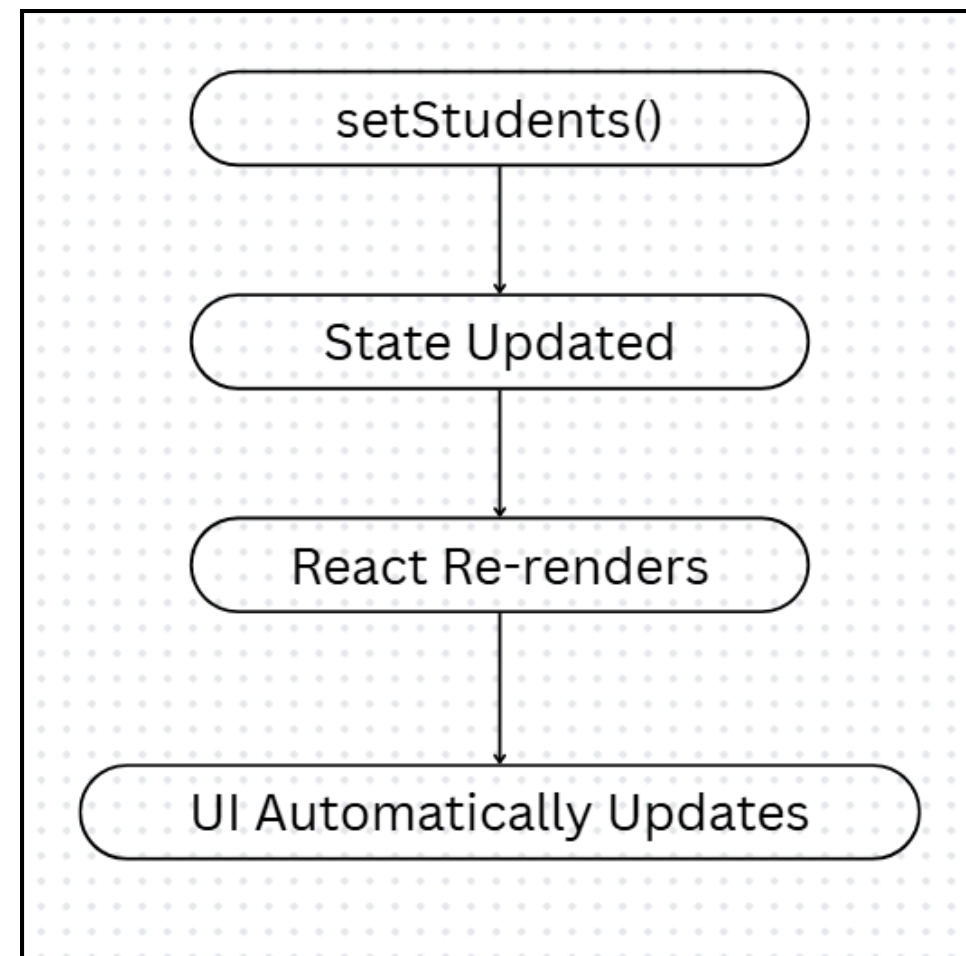
```
setStudents((prev) => [...prev, data]);
```

- **React adds the new student into existing state.**
 - Faster
 - Better UX
 - No page refresh

Data Flow in React

6. React Re-Render

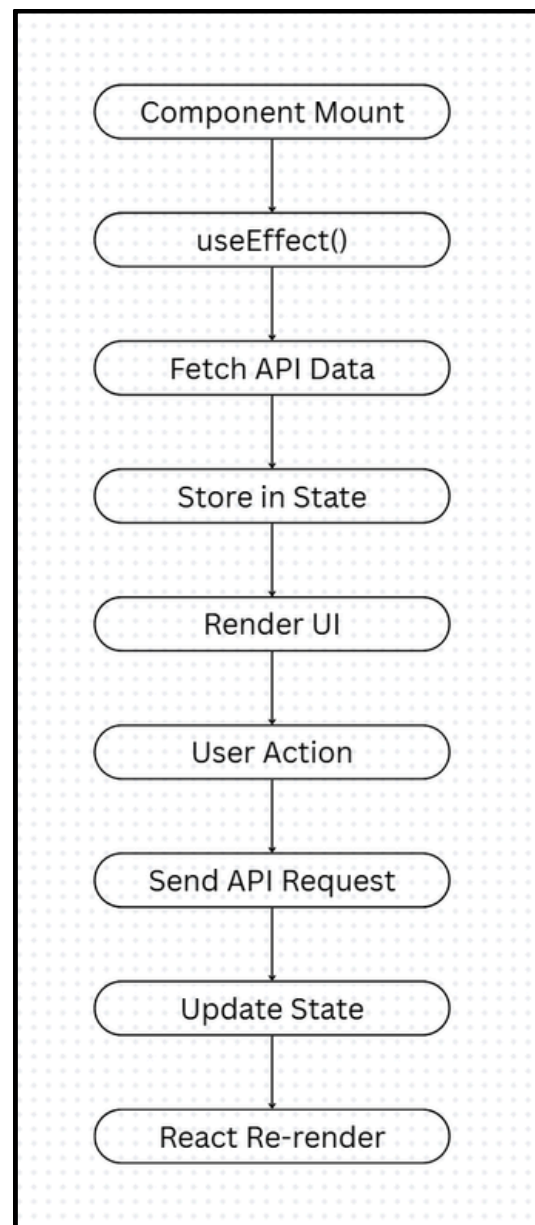
- When state changes:



ID	Name (Latin)	Gender	Year of Birth	Actions
1	Dalii	Female	2006	Edit Delete

Data Flow in React

Visual React Data Flow:



Important Concepts:

- React State = Single Source of Truth
 - UI always comes from state.
- **One-Way Data Flow**
 - State → Props → UI
- React Never Manually Updates HTML
 - React updates UI automatically after state changes.

Summary

- `useEffect()` loads data when component mounts
- `fetch()` gets data from API
- State stores application data
- Props send data to child components
- User actions trigger API calls
- Updating state causes re-render

What is CRUD?

CRUD is an acronym that represents the **four basic operations** used to manage data in a database.

CRUD Meaning

Letter	Operation	Description
C	Create	Add new data
R	Read	View or retrieve data
U	Update	Modify existing data
D	Delete	Remove data

What is CRUD?

CRUD is an acronym that represents the **four basic operations** used to manage data in a database.

Real-Life Example (Student System)

Operation	Example
Create	Add new student
Read	View student list
Update	Edit student name
Delete	Remove student

JavaScript Example (Array)

```
let students = [];  
  
// CREATE  
students.push("Dara");  
  
// READ  
console.log(students);  
  
// UPDATE  
students[0] = "Nita";  
  
// DELETE  
students.splice(0, 1);
```

What is CRUD?

CRUD is an acronym that represents the **four basic operations** used to manage data in a database.

API Example (HTTP Methods)

CRUD	HTTP Method	Example
Create	POST	Add new user
Read	GET	Get users
Update	PUT / PATCH	Update user
Delete	DELETE	Delete user

Why CRUD is Important?

- Core of all applications
- Used in APIs
- Used in databases
- Used in backend & frontend

CREATE (Insert Data)

What is CREATE?

- The **CREATE** operation is: **Adding new data (records) into a database**

CREATE → Add new data

CREATE = send data to server

HTTP Methods:

Operation	Method
UPDATE	PUT / PATCH

CREATE (Insert Data)

Example (Using Fetch API)

```
const createStudent = async (name, gender, yearOfBirth) => {
  try {
    const response = await fetch("https://api.example.com/students", {
      method: "POST",
      headers: {
        "Content-Type": "application/json"
      },
      body: JSON.stringify({
        name,
        gender,
        yearOfBirth
      })
    });

    const data = await response.json();
    console.log("Created:", data);
  } catch (error) {
    console.error("Error:", error);
  }
};
```

- **POST** → used to create data
- **fetch()** → send request to server
- **body** → data we want to insert
- **JSON.stringify()** → convert JS object → JSON
- **await** → wait for server response

Response

- `const data = await response.json();`
 data = newly created record

CREATE (Insert Data)

Example (Using Fetch API)

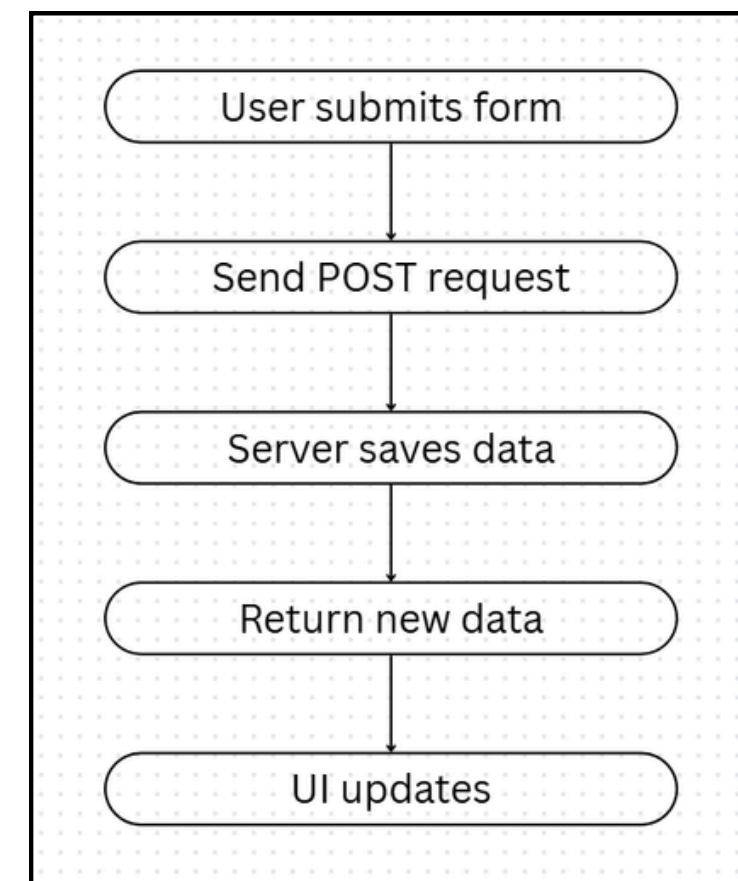
*Important Note:

- After CREATE:
 - fetch updated list again OR
 - update React state manually

Example (Update UI State)

```
setStudents(prev => [...prev, data]);
```

Real Data Flow:



Summary:

- CREATE = add new data
- Use POST method
- Send data using fetch()
- Update UI after creating

READ (Fetch Data)

What is READ?

- The **READ** operation is: Used to retrieve data from a database and display it to users

READ → Get Data

READ = get data from server and show it on screen

HTTP Methods:

Operation	Method
READ	GET

READ (Fetch Data)

Advanced READ (Filtering)

```
const fetchStudents = async () => {
  try {
    const response = await fetch("https://api.example.com/students");
    const students = await response.json();

    console.log("Students:", students);
    setStudents(students); // Store in React state
  } catch (error) {
    console.error("Error:", error);
  }
};
```

- `fetch()` → send GET request
- `GET` → default method (no need to specify)
- `await` → wait for response
- `response.json()` → convert JSON → JavaScript object

Store Data in State

- `Store Data in State` = `setStudents(students);`

Best Practice (useEffect)

```
useEffect(() => {
  fetchStudents();
}, []);
```

READ (Fetch Data)

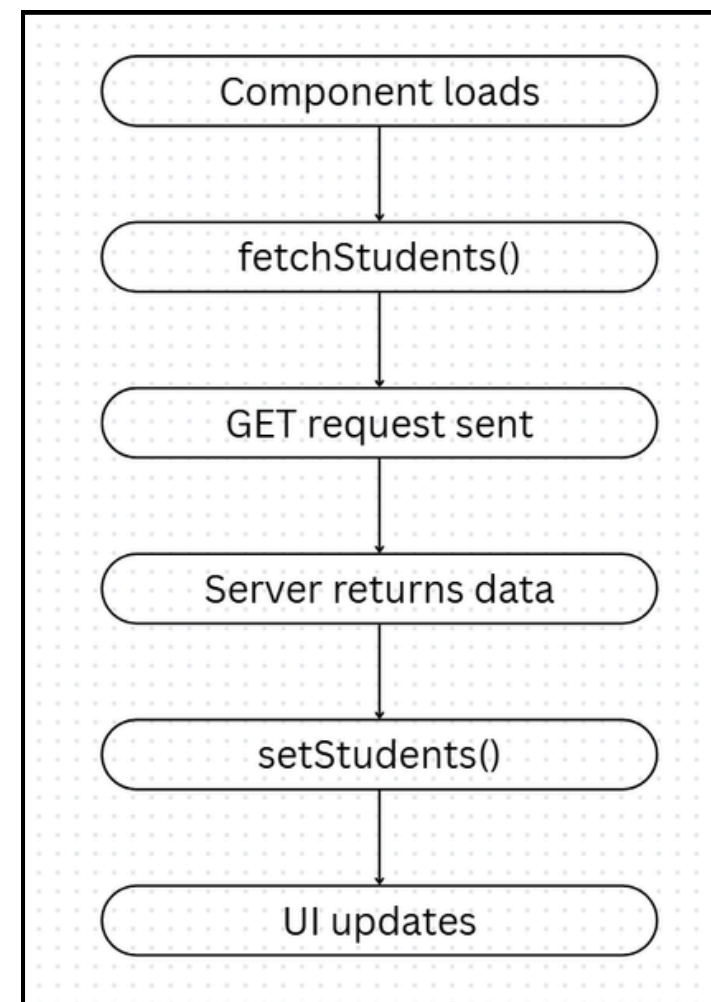
Advanced READ (Filtering)

- Example with Query Parameter

```
const loadStudents = async (gender) => {  
  try {  
    const response = await fetch(  
      `https://api.example.com/students?gender=${gender}`  
    );  
  
    const students = await response.json();  
    setStudents(students);  
  } catch (error) {  
    console.error("Error:", error);  
  }  
};
```

- `?gender=${gender}` → filter data
- similar to SQL: `WHERE gender = ?`

Real Data Flow:



Summary:

- READ = retrieve data
- use `fetch()` with GET
- store data in state
- use `useEffect` to load data
- can filter with query params

UPDATE (Modify Data)

What is UPDATE?

- The **UPDATE** operation is: Used to **modify existing data** without deleting it

UPDATE = modify existing data

NOT create new data

UPDATE = edit existing data

- Like:
 - edit profile
 - change password
 - update product

HTTP Methods

Operation	Method
UPDATE	PUT / PATCH

- **PUT** = replace entire data
- **PATCH** = update only some fields

UPDATE (Modify Data)

Example (Using Fetch API)

```
const updateStudent = async (studentId, updatedFields) => {
  try {
    const response = await fetch(
      `https://api.example.com/students/${studentId}`,
      {
        method: "PUT", // or PATCH
        headers: {
          "Content-Type": "application/json"
        },
        body: JSON.stringify(updatedFields)
      }
    );

    const data = await response.json();
    console.log("Updated:", data);

    fetchStudents(); // Refresh list
  } catch (error) {
    console.error("Error:", error);
    alert("Failed to update student");
  }
};
```

- **PUT / PATCH** → used to update data
- **studentId** → identify which record to update
- **body** → fields to update
- **JSON.stringify()** → convert object to JSON
- **await** → wait for response

Best Practice

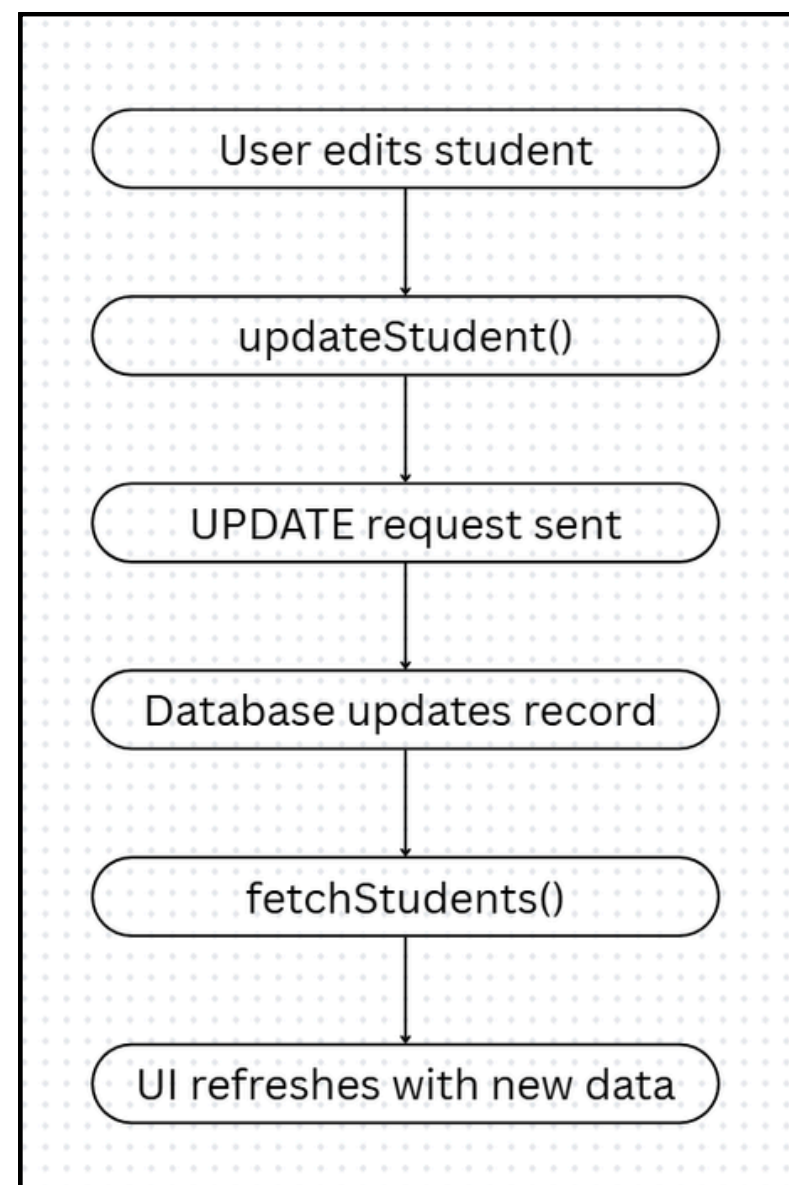
- Always specify **which record to update**
 - **students/1** → update student with ID = 1

React Tip

- After updating: **fetchStudents();**
 - refresh UI OR update state manually

UPDATE (Modify Data)

Real Data Flow:



Summary

- UPDATE = modify existing data
- use **PUT** or **PATCH**
- send updated fields in request body
- refresh UI after update

DELETE (Remove Data)

What is DELETE?

- The **DELETE** operation is: Used to **remove data permanently** from a database

DELETE is permanent (cannot undo)

DELETE = remove data forever

- Like:
 - delete student
 - delete product
 - delete message

HTTP Methods:

Operation	Method
DELETE	DELETE

DELETE (Remove Data)

Example (Using Fetch API)

```
const deleteStudent = async (studentId) => {
  try {
    const response = await fetch(
      `https://api.example.com/students/${studentId}`,
      {
        method: "DELETE"
      }
    );

    const data = await response.json();
    console.log("Deleted:", data);

    fetchStudents(); // Refresh the list
  } catch (error) {
    console.error("Error:", error);
    alert("Failed to delete student");
  }
};
```

- **DELETE** → HTTP method to remove data
- **studentId** → identifies which record to delete
- **fetchStudents()** → refresh UI after delete

Best Practice — Confirm Before Delete

- Always ask the user before deleting:

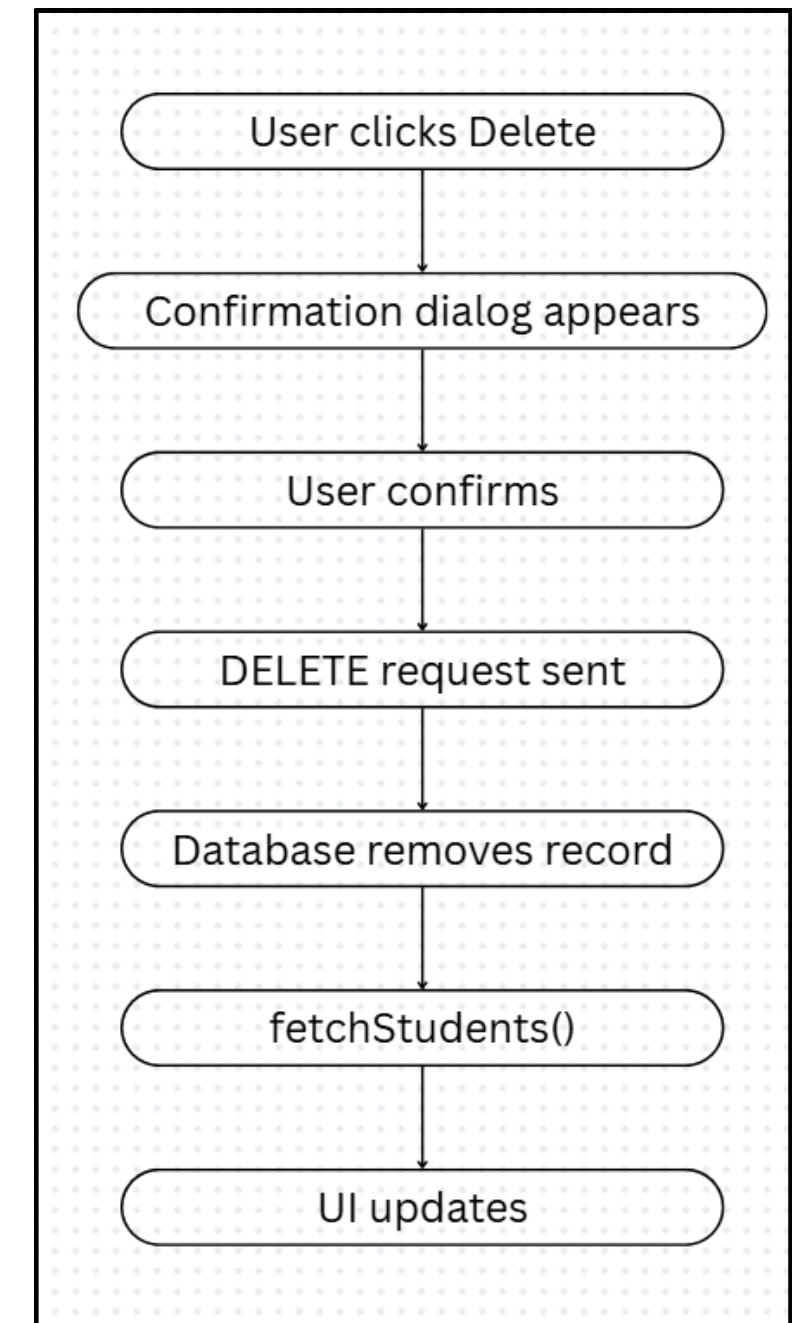
```
if (!window.confirm("Are you sure you want to delete this student?"))
{
  return;
}
```


DELETE (Remove Data)

Full Example with Confirmation

```
const deleteStudent = async (studentId) => {  
  if (!window.confirm("Are you sure you want to delete this student?")) {  
    return;  
  }  
  
  try {  
    const response = await fetch(  
      `https://api.example.com/students/${studentId}`,  
      {  
        method: "DELETE"  
      }  
    );  
  
    const data = await response.json();  
    console.log("Deleted:", data);  
  
    fetchStudents();  
  } catch (error) {  
    console.error("Error:", error);  
    alert("Failed to delete student");  
  }  
};
```

Real Data Flow:



Common Patterns & Best Practices

When building real-world React apps, following patterns and best practices helps you write **robust, maintainable, and user-friendly applications**.

We will use a **Student Management Example** with a public API:

<https://jsonplaceholder.typicode.com/users>

Common Patterns & Best Practices

1. Loading State

- Why?

- Users need feedback that data is being fetched. Without it, the interface feels frozen.

```
const [loading, setLoading] = useState(false);
const [students, setStudents] = useState([]);

const loadStudents = async () => {
  setLoading(true);
  try {
    const response = await fetch("https://jsonplaceholder.typicode.com/users");
    const data = await response.json();
    setStudents(data);
  } catch (error) {
    console.error("Error loading students:", error);
  } finally {
    setLoading(false);
  }
};
```

```
{loading ? (
  <p>Loading ...</p>
) : (
  <table>{/* Render students */}</table>
)}
```

Always give visual feedback during async operations.

Common Patterns & Best Practices

2. Error Handling

- **Why?**
 - Network issues, server errors, or permission problems can fail API requests. Your app should handle errors gracefully.

```
try {
  const response = await fetch("https://jsonplaceholder.typicode.com/users");
  if (!response.ok) throw new Error("Network response not ok");
  const data = await response.json();
  setStudents(data);
  setErrorMessage("");
} catch (error) {
  console.error("Error:", error);
  setErrorMessage(error.message);
}
```

```
{errorMessage &&
  <p className="text-red-500">
    {errorMessage}
  </p>
}
```

Common Patterns & Best Practices

3. Input Validation

- **Why?**
 - Validate user input before **sending it to the server** to prevent API errors.

```
if (!name || !gender || !yearOfBirth) {  
  alert("All fields are required!");  
  return;  
}  
  
const yearNum = parseInt(yearOfBirth);  
if (isNaN(yearNum) || yearNum < 1900 || yearNum > new Date().getFullYear()) {  
  alert("Please enter a valid year!");  
  return;  
}  
  
if (!["Male", "Female", "Other"].includes(gender)) {  
  alert("Please select a valid gender!");  
  return;  
}
```

Catching problems on the client side gives instant feedback and saves server resources.

Common Patterns & Best Practices

4. Confirmation Dialogs

- **Why?**
 - DELETE operations are permanent. Always confirm with the user.

```
const handleDelete = (id) => {  
  const confirmDelete = window.confirm("Are you sure you want to delete this student?");  
  if (!confirmDelete) return;  
  
  // Call API to delete  
};
```

Common Patterns & Best Practices

5. Form Reset

- Why?
 - After a successful create/update, reset the form to show submission success.

```
const handleSubmit = async (e) => {  
  e.preventDefault();  
  
  await createStudent({ name, gender, yearOfBirth });  
  
  setFormState({ name: "", gender: "Male", yearOfBirth: "" });  
};
```

*Prepares the form for **next entry** and gives visual feedback.*

Common Patterns & Best Practices

6. Unique Keys in Lists

- Why?
 - React uses **keys** to track items in a list. Without unique keys, React may **misrender** items.

```
<tbody>
  {students.map((s) => (
    <tr key={s.id}>
      <td>{s.id}</td>
      <td>{s.name}</td>
      <td>{s.gender}</td>
      <td>{s.yearOfBirth}</td>
    </tr>
  ))}
</tbody>
```


Common Patterns & Best Practices

7. Error Boundary Pattern

- **Why?**
 - Prevent the whole app from crashing due to unexpected component errors.

```
try {  
  return <StudentsTable students={students} />;  
} catch (error) {  
  console.error("Component error:", error);  
  return <div>Something went wrong. Please refresh the page.</div>;  
}
```

Provides graceful fallback in production apps.

Common Patterns & Best Practices

8. Debouncing Search/Filter

- **Why?**
 - Avoid sending an API request **on every keystroke**.

```
const [searchTerm, setSearchTerm] = useState("");

const handleSearch = (term) => {
  setSearchTerm(term);
  setTimeout(() => {
    fetchByName(term); // API call
  }, 500); // Delay 500ms
};
```

Reduces server load and improves UX.

Common Patterns & Best Practices

9. Loading Skeleton

- **Why?**
 - Instead of generic "Loading..." messages, show the **shape of content**.

```
{loading ? (  
  <div className="skeleton-table">  
    <div className="skeleton-row"></div>  
    <div className="skeleton-row"></div>  
    <div className="skeleton-row"></div>  
  </div>  
) : (  
  <table>{/* Render students */}</table>  
)}
```

Skeletons improve perceived performance.

Common Patterns & Best Practices

10. Empty State Handling

- **Why?**
 - Show helpful messages when no data exists.

```
{students.length === 0 ? (  
  <p>No students found. Add one to get started!</p>  
) : (  
  <table>{/* Render students */}</table>  
)}
```

Prevents confusion and guides the user.



THANK YOU

For your attention !

